

Demo Script

Playwright Beyond the Happy Path

The stage script for the talk deck `playwright-beyond-the-happy-path-dnug-ka-2026-09-24` — 21 slides. The demos run as **6 continuous sessions**. Each session keeps its terminal and its browser open, and each numbered step is one slide. The number in brackets is the slide number.

The sessions together need about **48 minutes**. This leaves about 10 minutes for questions in the 60 minute slot. Do the preparation before the talk starts. All paths are relative to `src/deploy-or-die-frontend`, if no other path is given.

The content pool `playwright-content-pool` holds all 30 slides. The 9 slides that are not in the talk deck have no step here.

Preparation — 10 min, before the talk

1. Start the containers one time, so that the image build is not on stage: `pnpm run test:integration`. Keep the report — you need it in session B.
2. Make sure that `ACCESSIBLE_MODE` is `false` in `src/a11yMode.ts`. The accessibility demo needs this.
3. Start the dev server: `pnpm run dev`. It must stay up for all sessions.
4. Install the browser for the C# tests one time: `pwsh bin/Debug/net10.0/playwright.ps1 install chromium` in `src/deploy-or-die-dotnet-tests/DeployOrDie.E2E.Tests`.
5. Open three terminals: dev server, test runner, and one spare.

Watch out: the container build needs about 4 minutes. Never do it on stage.

Session 0 · Frame — Slides 2–3 · 3 min

1. **[2]** Say who you are in three sentences. The slide holds the rest.
2. **[3]** Show the app: the name field, the deploy button and the result message. Name the two versions in the table — 1.63.0 for TypeScript and 1.62.0 for .NET.
3. Deploy one time and let the audience see the 30 second cooldown.

Watch out: the audience must see the cooldown here. Slide 15 shows the C# test that moves this cooldown forward with the Clock API.

Session A · Tools and traces — Slides 4–8 · 11 min

1. **[4]** `pnpm run test:ui` — watch mode, pick a locator with the mouse, time travel. Keep this short. It is a quick win, not the subject of the talk.
2. **[5]** Show the two `testDir` lines. Name the two effects from that project: the slow CI job, and the slow start of UI mode.
3. **[6]** `npx playwright show-trace test-results/*/trace.zip` — step back in time, then change to the ARIA tab. The ARIA tab is new in 1.63.
4. **[7]** Walk the two workflow steps. The report goes up after each run, the trace and the video only after a failure.
5. **[8]** Tell the story of the team that wrote mostly Playwright tests. Close with the line on the slide: an unreliable E2E test is worse than no E2E test.

Watch out: close UI mode before session B. It keeps a browser and a port.

Session B · Mock or real backend — Slides 9–13 · 13 min

1. **[9]** `pnpm run test:mocked` — 17 tests in about 10 seconds, with no backend. One test is red. Say that this is on purpose, and that session C explains it.
2. **[10]** `pnpm run test:integration` — the containers are warm, then 4 tests run against the real ASP.NET Core API and PostgreSQL.
3. **[11]** Show the `healthcheck` and `depends_on` in the Compose file. Name the three things that broke when you started each container from the test code.
4. **[12]** Go through the three questions. Give the answers for the demo app: one service, a small contract.
5. **[13]** Tell what happened to the team that released an API that was not compatible. Then show the three options for the pipeline.

Watch out: do not start the containers again here. The build takes about 4 minutes, and the warm containers from the preparation are enough.

Session C · Accessibility — Slide 14 · 5 min

1. **[14]** `pnpm run test:a11y` — red. Three rules fail: `button-name`, `color-contrast` and `label`.
2. `pnpm run test:report` — show the attached `axe-results.json`.
3. Set `ACCESSIBLE_MODE = true` in `src/a11yMode.ts`. Run again — green. This is the red to green moment of the talk.
4. Say the limit out loud: an automatic scan finds 30% to 50% of the problems. It does not replace a test with a screen reader.
5. Set the value back to `false`.

Watch out: set `ACCESSIBLE_MODE` back to `false` after the talk, or the demo is gone the next time.

Session D · .NET — Slides 15–17 · 9 min

1. **[15]** The dev server from the preparation must still run on port 3000. `dotnet test` — 5 tests, 1 red. Put `DeployTests.cs` and `tests/cooldown.spec.ts` side by side. Same locators, same auto-wait, same Clock API. Only the names use the .NET style.
2. **[16]** Walk the two boxes. Say the one sentence that matters: the browser API is the same, the test runner is not. Say in one sentence what a test lock and an isolated retry are — this deck does not show them, but the box names them. Then run `npx playwright codegen --target csharp-xunit http://localhost:3000` and record one click — the recorder writes C#.
3. **[17]** Show the red `AccessibilityTests.cs`. Both languages report the same three rules: `button-name`, `color-contrast` and `label`. Put the C# output next to the TypeScript output from session C.

Watch out: `global.json` selects the Microsoft Testing Platform runner. Without it, `dotnet test` fails on the .NET 10 SDK. The C# tests need their own browser: `pwsh bin/Debug/net10.0/playwright.ps1 install chromium`.

Session E · AI and agents — Slides 18–19 · 5 min

1. **[18]** `npx playwright mcp` — the server is part of Playwright since 1.62.
2. Give the agent one task: explore `http://localhost:3000`. Let it find the cooldown with no hint from you.
3. **[19]** Show the three defects in the wrong example on the slide. Open `tests/locators.spec.ts` next to it. Close with the rule: the agent explores, the human reviews.

Watch out: this session needs network and a model. Record a fallback video before the talk.

Close — Slides 20–21 · 2 min

1. **[20]** Read the two lessons. They are the two that the six slides from real projects did not already make.
2. **[21]** Leave the last slide up for the questions. The QR code goes to the repository with the demo project and these slides.

Watch out: if you are late, drop step [20], then slide 17, then slide 6. Do not drop a slide from a real project — they are the reason for this talk.

Command reference

SESSION	COMMAND
Preparation	<code>pnpm run test:integration && pnpm run dev</code>
A · Tools and traces	<code>pnpm run test:ui</code>
A · Tools and traces	<code>npx playwright show-trace test-results/*/trace.zip</code>
B · Mock or real	<code>pnpm run test:mocked · pnpm run test:integration</code>
C · Accessibility	<code>pnpm run test:a11y · pnpm run test:report</code>
D · .NET	<code>dotnet test (in src/deploy-or-die-dotnet-tests/DeployOrDie.E2E.Tests) — 5 tests, 1 red</code>

SESSION	COMMAND
D · .NET	npx playwright codegen --target csharp-xunit http://localhost:3000
E · AI and agents	npx playwright mcp

